

Guided Practice: MVC Application Structure

Outcome

You will create a Node and Express web application that uses and runs on the MEAN stack and is implemented using the MVC architecture.

Resources Needed

- VCASTLE
- OR --
- System configured for MEAN stack development.

Level of Difficulty

Moderate

Deliverables

Deliverables are marked in red font or with a red picture border around the screenshot. **Your name should be visible as directed in screenshots that you submit.**

You will submit a link to your GitHub repository that contains files associated with this assignment. This will include:

1. Screenshot of the application running.
2. Committed code files.

Program Specifications

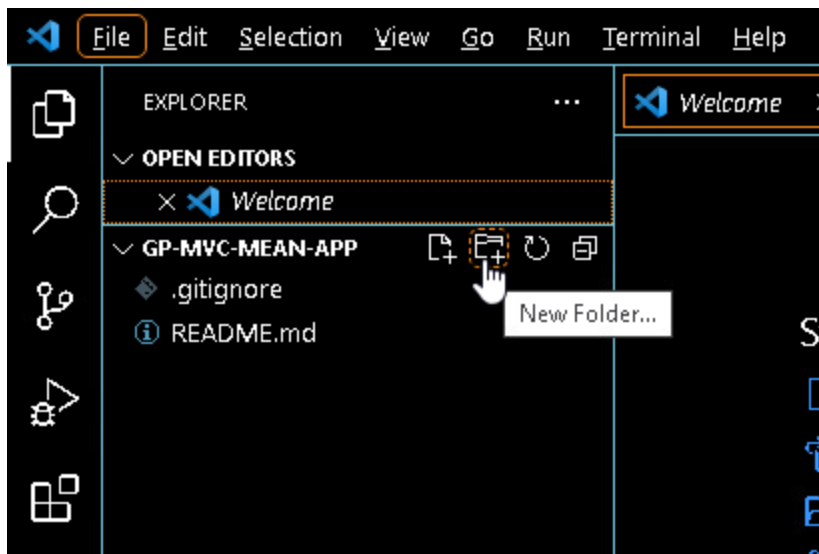
This practice program is designed to demonstrate and help you gain a deeper understanding of creating and running an MVC-architected MEAN stack web application.

Development Steps

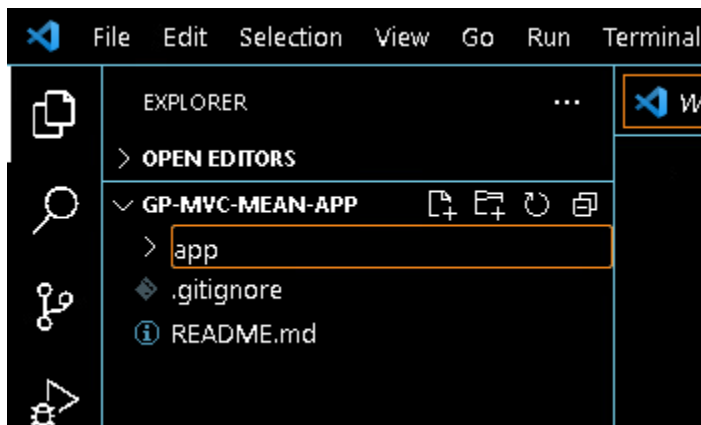
Create the Folder Structure

In this part of the guided practice, you will establish the folder structure used for application architecture. This establishes a “layered” architecture with separate folders for the model (data-related), view (UI-related), and controller (linkage between the UI and the data) files. This can be accomplished in many ways - creating the folders directly in the repo in GitHub before cloning (or after cloning and pulling the changes), creating the folders using Windows explorer, using a command prompt, etc. This technique is used because it keeps all the work and focus inside of VS Code.

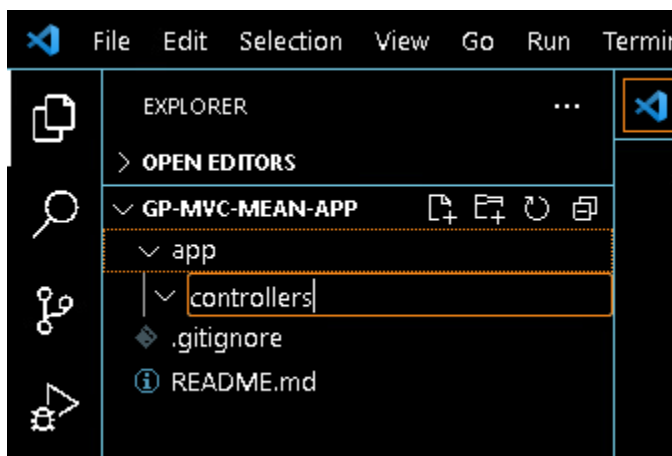
1. Click the “New Folder...” icon in the Folders panel.



2. Type "app" and press Enter to create a folder named "app". Follow the casing as shown in these instructions.

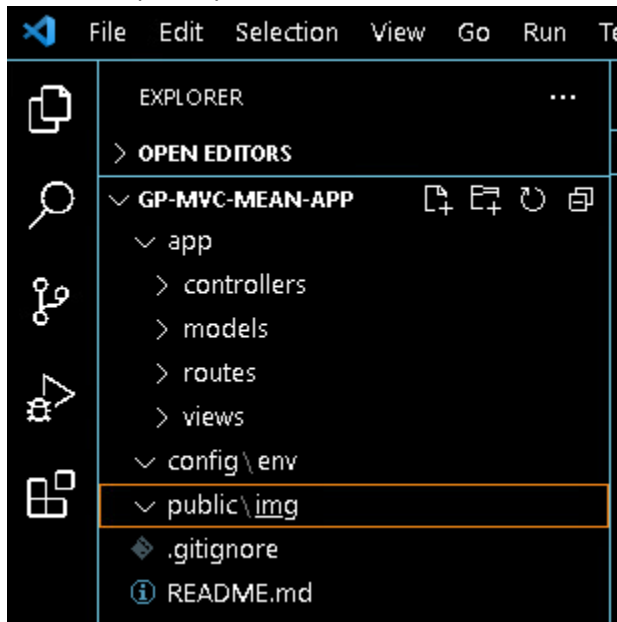


3. With the app folder selected, click the New Folder icon and add the folder "controllers" inside the app folder.



4. Repeat this process to add the following folders to the app folder:
 - a. models
 - b. routes
 - c. views

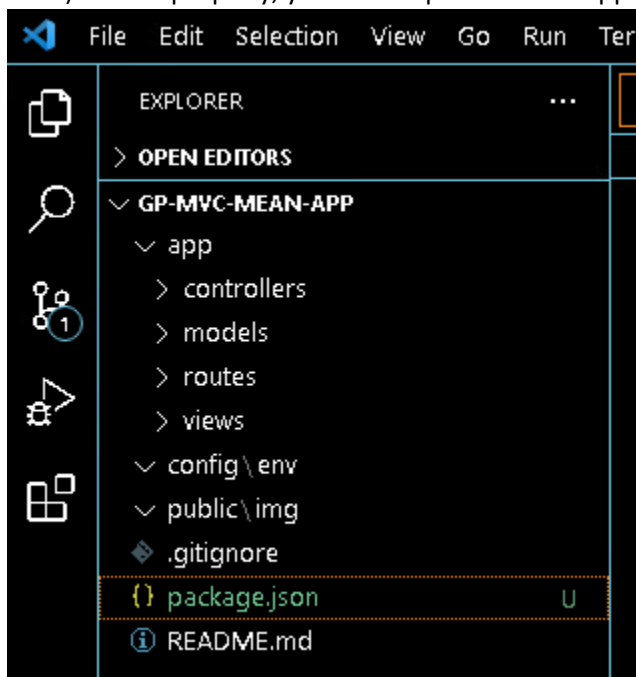
- Click outside of the folder (in the open area) or click the .gitignore or README.md file, then click the New Folder icon. Name the folder “config”. This folder will be at the project root level.
- Inside the config folder, add a folder named “env”.
- Create a folder named “public” at the root project level (same level as app and config).
- Inside the public folder, add a folder named “img”.
- When complete, your folder structure should look as follows:



Create & Run the Initial Application:

In this section of the GP, you will add the files needed to create a basic web application that displays the message “Hello <Your Name>”.

- In the application root folder, add a file named “package.json”. (The process for adding a file is very similar to that of adding a folder but done by clicking on the New File icon rather than the New Folder icon.) If done properly, your folder panel should appear as follows:



Add the following code to the package.json file:

```
{
  "name": "mean-mvc",
  "version": "0.0.1",
  "dependencies": {
    "express": "4.18.2"
  }
}
```

2. In the app > controllers folder, add a file named "index.server.controller.js". Add the following code to the index.server.controller.js file:

```
exports.render =
  function (req, res) {
    res.status(200).send('Hello, your name!');
  };

```

3. In the app > routes folder, add a file named "index.server.routes.js". Add the following code to the index.server.routes.js file:

```
module.exports =
  function (app) {
    const index = require('../controllers/index.server.controller');
    app.get('/', index.render);
  };

```

4. In the config folder, add a file named "express.js". Add the following code to the express.js file:

```
const express = require('express');
module.exports = function() {
  const app = express();
  require('../app/routes/index.server.routes')(app);
  return app;
};

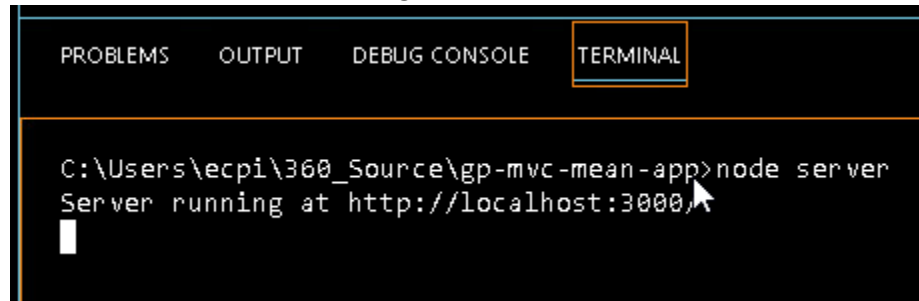
```

5. In the application root folder, add a file named "server.js". Add the following code to the server.js file:

```
const configureExpress = require('./config/express');
const app = configureExpress();
app.listen(3000);
module.exports = app;
console.log('Server running at http://localhost:3000/');
```

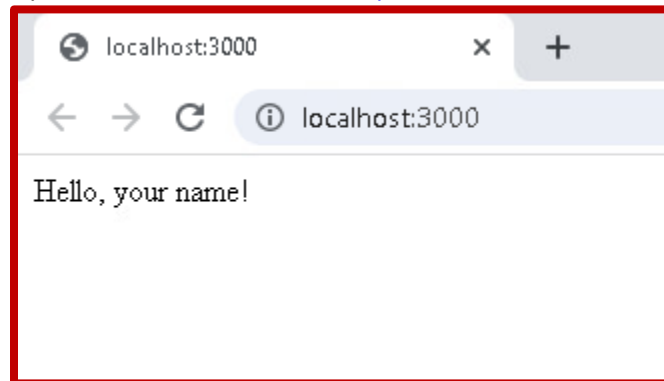
6. We're now ready to run the application. To run the app, do the following:
 - a. Open a command (cmd) terminal (this was covered in the first guided practice) in VS Code.
 - b. Run the command (type the command and press enter):
npm install
 - c. Once complete (it may take a while to complete), start the server by running the command:
node server

This should result in the following:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
C:\Users\ecpi\360_Source\gp-mvc-mean-app>node server
Server running at http://localhost:3000/
```

- d. Open a browser window to <http://localhost:3000/> to see the Hello message in the response.



Take a screenshot that resembles the one above and save it in the same folder as your README.md file in the repository. Your name should be visible in the screenshot.

Add Configuration, EJS View, and Session Support

In this section, we will add support to configure the application for running in development mode or production mode. This will include adding support for reading environment variables and adding a development configuration. In addition to the configuration modifications, we will add support for having a view (UI) that supports the ejs view type. We will also add session tracking support to the application.

Note that while we saw a message displayed in the browser at the end of the previous section, what we were seeing was simply the response from the server code displayed by a browser - there was no HTML or other code to format that response. The browser was simply displaying the raw response from the server.

1. First, we'll add support to our application for reading from environment variables and displaying information using ejs. Modify the package.json file created above by adding the following packages:
 - body-parser
 - compression
 - method-override
 - morgan
 - ejs
 - express-session

The final code should appear as follows (this includes the code from the previous section). Note that by convention the dependencies are listed in alphabetical order:

```
{
  "name": "mean-mvc",
  "version": "0.0.1",
  "dependencies": {
    "body-parser": "1.20.2",
    "compression": "1.7.4",
    "ejs": "3.1.9",
    "express": "4.18.2",
    "express-session": "1.17.3",
    "method-override": "3.0.0",
    "morgan": "1.10.0"
  }
}
```

- Next, we'll add reading of the environment configuration to the server.js file. This includes adding a single line to the top of the server.js file. Note that we include a default value of "development" in the event that the value of NODE_ENV is not set. The final code should appear as follows (this includes the code from the previous section):

```
process.env.NODE_ENV = process.env.NODE_ENV || 'development';
const configureExpress = require('./config/express');
const app = configureExpress();
app.listen(3000);
module.exports = app;
console.log('Server running at http://localhost:3000/');
```

- Now we'll add a development configuration that includes a secret key to sign the session identifier. In the config > env folder, add a file named "development.js". Add the following code to the development.js file:

```
module.exports = {
  //development configuration options
  sessionSecret: 'developmentSessionSecret'
};
```

- Next we'll add a file to configure the proper require statement based on the configuration. The code uses string concatenation to load the proper config file. In the config folder, add a file named "config.js". Add the following code to the config.js file:

```
//if process.env.NODE_ENV is set to production, it reads from
//the production.js file; if set to development it reads from
//the development.js file
module.exports = require('./env/' + process.env.NODE_ENV);
```

- Now we'll modify the config > express.js file to use the configuration reading modules, use EJS, serve static files, and include the session package. This includes a significant number of updates to the file. The final code should appear as follows (this includes the code from the previous section):

```

const config = require('./config');
const express = require('express');
const morgan = require('morgan');
const compress = require('compression');
const bodyParser = require('body-parser');
const methodOverride = require('method-override');
const session = require('express-session');

module.exports = function() {
  const app = express();
  if (process.env.NODE_ENV === 'development') {
    app.use(morgan('dev'));
  } else if (process.env.NODE_ENV === 'production') {
    app.use(compress());
  }

  app.use(bodyParser.urlencoded({ extended: true }));
  app.use(bodyParser.json());
  app.use(methodOverride());
  app.use(session({
    saveUninitialized: true,
    resave: true,
    secret: config.sessionSecret
  }));
  app.set('views', './app/views');
  app.set('view engine', 'ejs');
  app.use(express.static('./public'));
  require('./app/routes/index.server.routes')(app);
  return app;
};

```

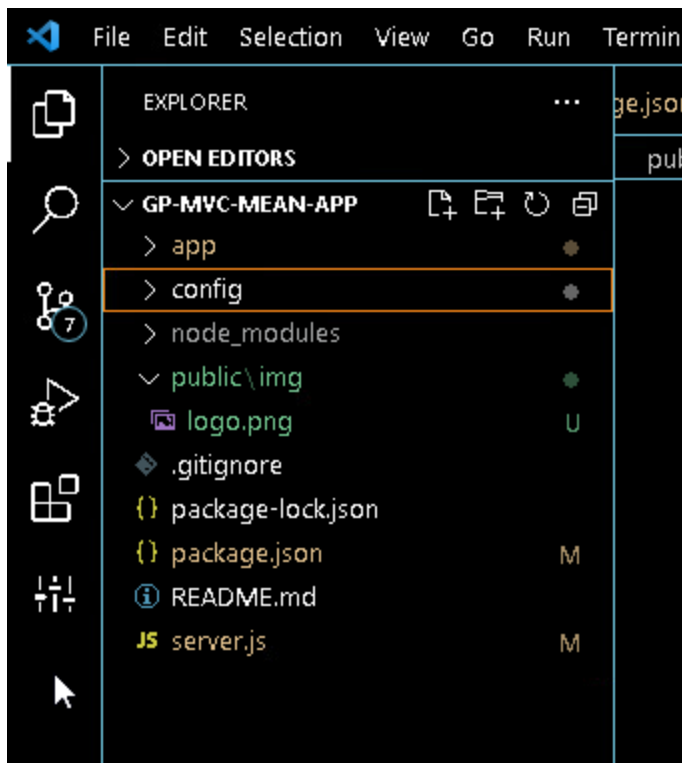
- Now we'll modify the app > controllers > index.server.controller.js file to store and log session information and to render the index file (index.ejs created in the next step), passing a title to the index file. The final code should appear as follows (this includes the code from the previous section):

```

exports.render =
  function (req, res) {
    if (req.session.lastVisit) {
      console.log(req.session.lastVisit);
    }
    req.session.lastVisit = new Date();
    res.render('index', { title: 'Hello your name' });
  };

```

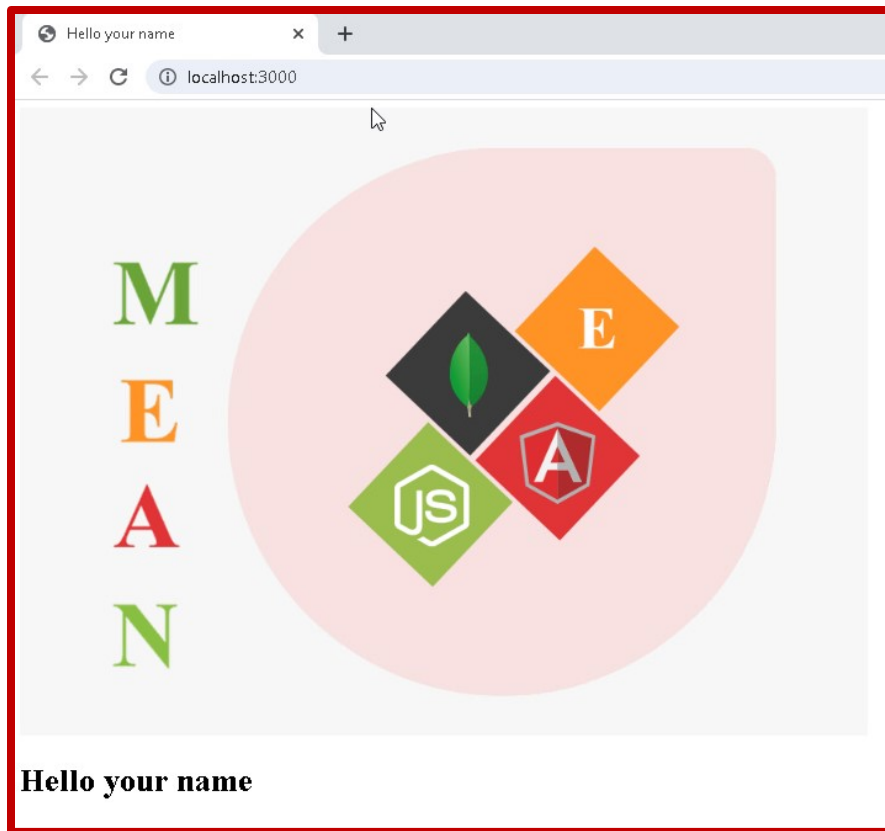
- Now we'll add a view file to display the output from the server.controller file. The page we're creating will include an image, so, to begin, copy an image file of your choosing into the public > img folder.



In the app > views folder, add a file named “index.ejs”. Add the following code to the index.ejs file, noting that you will need to replace “logo.png” in the code below with the name of the file you placed in the public > img folder:

```
<!DOCTYPE html>
<html>
  <head>
    <title><%= title %></title>
  </head>
  <body>
    
    <h1><%= title %></h1>
  </body>
</html>
```

8. We're now ready to run the application. To run the app, do the following:
 - a. Open a command (cmd) terminal (this was covered in the first guided practice) in VS Code.
 - b. Run the command (type the command and press enter - this is necessary because we modified package.json, so the dependencies have changed):
npm install
 - c. Once complete (it may take a while to complete), start the server by running the command:
node server
 - d. Open a browser window to <http://localhost:3000/> to see your image and message displayed (remember the message comes across as the variable “title” from index.server.controller.js).



Take a screenshot that resembles the one above and save it in the same folder as your README.md file in the repository. Your name should be visible in the screenshot.

- e. Refresh the page in the browser. View the terminal window in VS Code and note the printout of the date-time string. This is the session working, executing line 4 in index.server.controller.js. When the page loaded, the session.lastVisit was set to the current date/time. On refresh, that value is logged to the console:

```
{ } package.json M    JS index.server.controller.js M X    <> index.ejs U
app > controllers > JS index.server.controller.js > ...
1  exports.render =
2      function (req, res) {
3          if (req.session.lastVisit) {
4              console.log(req.session.lastVisit);
5          }
6          req.session.lastVisit = new Date();
7          res.render('index', { title: 'Hello your name' });
8      };
9

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
C:\Users\ecpi\360_Source\gp-mvc-mean-app>node server
Server running at http://localhost:3000/
GET / 304 49.137 ms - -
GET /img/logo.png 304 3.811 ms - -
GET /favicon.ico 404 3.767 ms - 150
2023-05-19T14:31:48.799Z
GET / 304 8.164 ms - -
GET /img/logo.png 304 22.434 ms - -
```